

DOSSIER RÉSUMÉ

GROUPE 1 — LOCATION DE FILMS

Projet Bases de données sur la base Sakila (MySQL / MariaDB)

Membres : Esther Lydie · Jude · Lidvige Johanne · Melchior Junior · Oniandji · Terrence · Kerane

Encadrant : Prime Clet ESSE ESSE

Soutenance : 23 juillet 2026

Thème : rental · inventory · customer · film

1. Présentation du thème

Notre thème couvre le cœur de métier d'un vidéoclub : la **location de films**. Les quatre tables principales sont **rental** (les locations), **inventory** (les exemplaires physiques), **customer** (les clients) et **film** (le catalogue). La question métier centrale qui traverse nos quatre modules est : « *cet exemplaire est-il disponible ?* » — c'est elle que nos vues affichent, que notre trigger protège, que notre transaction sécurise en concurrence et que notre index composite accélère.

2. Module 7 — Vues, procédure stockée et triggers

Nous avons créé deux vues. **v_locations_en_cours** liste les exemplaires non rendus avec la date de retour prévue, calculée à partir de **rental_duration**. **v_historique_locations** est notre **vue de sécurité** : elle joint location, client et film mais masque volontairement l'email, l'adresse et les montants ; c'est la seule porte d'entrée de l'utilisateur restreint du module 11.

La procédure **enregistrer_location(client, film, magasin, employé)** vérifie que le client est actif, cherche et verrouille un exemplaire libre, écrit la location et retourne la date de retour prévue calculée.

Le trigger **trg_bloque_double_location** (BEFORE INSERT sur **rental**) refuse de louer un exemplaire déjà sorti et journalise la tentative dans **audit_location**. Cette table d'audit est en **MyISAM**, moteur non transactionnel : la ligne d'audit survit ainsi à l'annulation de l'INSERT refusé — avec InnoDB elle serait annulée avec lui. Un second trigger journalise les locations validées.

3. Module 8 — Transactions, ACID et concurrence

La procédure **louer_et_encaisser** regroupe deux écritures (location dans **rental**, paiement dans **payment**) dans une transaction : si une étape échoue, un handler exécute **ROLLBACK** et la base reste intacte — le « tout ou rien ».

ACID en une phrase chacune. *Atomicité* : la location et le paiement réussissent ou échouent ensemble, jamais à moitié. *Cohérence* : la base passe d'un état valide à un autre état valide, sans paiement orphelin. *Isolation* : deux guichets simultanés ne voient pas les opérations inachevées l'un de l'autre. *Durabilité* : après COMMIT, la location survit même à une coupure de courant.

Concurrence. Nous mettons en scène deux guichets louant le dernier exemplaire au même instant. Sans protection, les deux vérifications de disponibilité passent (sous REPEATABLE READ, la session B ne voit pas l'INSERT non validé de A — le trigger lui-même est aveugle à cette écriture) et la **double location** est enregistrée. La solution : verrouiller l'exemplaire avec **SELECT ... FOR UPDATE** dans une transaction ; la deuxième session attend, constate après le COMMIT de la première que l'exemplaire est pris, et renonce. L'anomalie disparaît.

4. Module 9 — Indexation et optimisation

Point méthodologique : Sakila installe déjà des index sur rental (exigés par les clés étrangères). Pour une mesure honnête de l'« avant », nous retirons d'abord ces index (et les FK qui s'y appuient), mesurons, créons nos index, puis remettons les FK.

Requêtes mesurées (protocole : **SET profiling = 1**, chaque requête exécutée 3 fois, moyenne retenue) :

Requête	Temps avant (ms)	Plan avant	Temps après (ms)	Plan après	Gain
R1 — Locations d'un client (customer_id = 148)	8,13	ALL — 16 005 lignes	1,02	ref idx_rental_customer	≈ 8×
R2 — Exemple sorti ? (inventory_id = 367 AND return_date IS NULL)	7,38	ALL — 16 005 lignes	0,55	ref idx_loc_inventaire_retour	≈ 13×
R3 — Locations non rendues (jointure 4 tables)	8,24	ALL sur rental	2,27	ref (jointures indexées)	≈ 3,6×
R4 — Historique d'un exemplaire	8,61	ALL	0,81	ref idx_loc_inventaire_retour	≈ 10,6×

Mesures obtenues avec SET profiling = 1 / SHOW PROFILES sur MariaDB 10.4.32, moyenne de 3 exécutions pour R1 et R2 (une seule exécution mesurée pour R3 et R4). Base : Sakila avec ~16 000 locations.

Index créés : `idx_rental_customer (customer_id)` (index simple, colonne du WHERE de R1) et `idx_loc_inventaire_retour (inventory_id, return_date)` (index **composite** : colonne d'égalité en premier, filtre ensuite ; il couvre R2 et R4 et accélère le trigger et la procédure). La cause de la lenteur initiale est le **parcours complet de table** (type ALL dans EXPLAIN) : sans index, la base lit les 16 044 lignes pour en retenir quelques dizaines.

5. Module 11 — Sécurité et administration

L'utilisateur `guichet` illustre le **principe du moindre privilège** : SELECT sur les deux vues, EXECUTE sur `enregistrer_location`, et rien d'autre.

Objet	root (admin)	guichet
Tables rental, customer, film, inventory	Tous droits	Aucun accès (SELECT refusé)
v_locations_en_cours	Tous droits	SELECT seulement
v_historique_locations	Tous droits	SELECT seulement
enregistrer_location	Tous droits	EXECUTE seulement
DELETE / UPDATE / DROP	Oui	Refusé
Email, adresse, montants	Visibles	Jamais visibles

Connecté en `guichet` : `SELECT * FROM customer` est refusé ; les vues fonctionnent mais sans colonnes sensibles ; DELETE échoue ; la location reste possible via la procédure (exécutée avec les droits de son créateur), voie d'écriture unique, contrôlée et audité.

Sauvegarde / restauration. Sauvegarde par `mysqldump --routines --triggers sakila > sakila_sauvegarde.sql` ; sinistre simulé par `DELETE FROM payment` (0 ligne restante) ; restauration par `mysql sakila < sakila_sauvegarde.sql` ; vérification par comptage : l'état initial est rétabli (16 044 paiements).

6. Conclusion

Le fil conducteur du projet est la disponibilité d'un exemplaire : les vues la montrent, le trigger la protège contre les erreurs, la transaction avec verrou la protège contre la concurrence, l'index composite la rend instantanée à vérifier, et la politique de droits garantit que seuls les canaux prévus peuvent la modifier. Tous les scripts s'exécutent tels quels sur une base Sakila vierge.

Annexe — Scripts SQL complets

Les six scripts ci-dessous s'exécutent dans l'ordre sur une base Sakila neuve (schéma + données officiels). Le premier installe tous les objets (vues, procédures, triggers, index, utilisateur restreint) ; les suivants servent de démonstration pour la soutenance.

A.1 — 01_installation.sql

Installation complète : table d'audit, vues, procédures stockées, triggers, index composite et utilisateur restreint « guichet ». Script rejouable (nettoyage en tête).

```
-- =====
-- PROJET BASES DE DONNEES – GROUPE 1 : LOCATION DE FILMS
-- Tables principales : rental, inventory, customer, film
-- Fichier : 01_installation.sql
-- A executer sur une base Sakila NEUVE (MySQL / MariaDB).
-- Cree : table d'audit, vues, procedures stockees, triggers,
--       index composite et utilisateur restreint "guichet".
-- Le script est RE-EXECUTABLE (nettoyage en tete).
-- phpMyAdmin : coller tel quel dans l'onglet SQL (les lignes
-- DELIMITER sont comprises par phpMyAdmin).
-- =====

USE sakila;

-- -----
-- 0. NETTOYAGE (permet de rejouer le script sans erreur)
-- -----

DROP TRIGGER IF EXISTS trg_bloque_double_location;
DROP TRIGGER IF EXISTS trg_audit_location_ok;
DROP PROCEDURE IF EXISTS enregistrer_location;
DROP PROCEDURE IF EXISTS louer_et_encaisser;
DROP VIEW IF EXISTS v_locations_en_cours;
DROP VIEW IF EXISTS v_historique_locations;
DROP TABLE IF EXISTS audit_location;
DROP USER IF EXISTS 'guichet'@'localhost';

-- -----
-- 1. TABLE D'AUDIT (Module 7)
-- -----

-- POURQUOI ENGINE=MyISAM ? MyISAM est NON transactionnel :
-- quand le trigger refuse une location (SIGNAL), l'INSERT fautif
-- est annule, mais la ligne d'audit ecrite par le trigger
-- SURVIT a cette annulation. Avec InnoDB, elle serait annulee
-- en meme temps que l'instruction refusee.
CREATE TABLE audit_location (
  audit_id INT AUTO_INCREMENT PRIMARY KEY,
  action VARCHAR(30) NOT NULL, -- LOCATION_OK / LOCATION_REFUSEE
  inventory_id INT NULL,
  customer_id INT NULL,
  utilisateur VARCHAR(100) NOT NULL, -- qui a fait l'action (CURRENT_USER)
  date_action DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  message VARCHAR(255) NULL
) ENGINE = MyISAM
COMMENT = 'Journal des locations : tentatives refusees et locations validees';

-- -----
```

```
-- 2. VUES (Module 7)
-- -----

-- Vue 1 : LOCATIONS EN COURS (exemplaires non rendus).
-- Vue "metier" du guichetier : quel film, quel client, quel
-- magasin, date de retour prevue (calculee avec rental_duration).
CREATE VIEW v_locations_en_cours AS
SELECT  r.rental_id,
        r.rental_date                                AS date_location,
        DATE_ADD(r.rental_date, INTERVAL f.rental_duration DAY) AS retour_prevu,
        f.title                                       AS film,
        i.store_id                                   AS magasin,
        CONCAT(c.first_name, ' ', c.last_name)      AS client
FROM    rental    r
JOIN    inventory i ON i.inventory_id = r.inventory_id
JOIN    film      f ON f.film_id      = i.film_id
JOIN    customer  c ON c.customer_id  = r.customer_id
WHERE   r.return_date IS NULL;

-- Vue 2 : HISTORIQUE LOCATION + CLIENT + FILM – VUE DE SECURITE.
-- Colonnes sensibles volontairement MASQUEES : ni email, ni
-- adresse du client, ni montants. C'est la SEULE porte d'entree
-- de l'utilisateur restreint "guichet" (Module 11).
CREATE VIEW v_historique_locations AS
SELECT  r.rental_id,
        r.rental_date AS date_location,
        r.return_date AS date_retour,
        f.title       AS film,
        f.rating       AS classification,
        CONCAT(c.first_name, ' ', c.last_name) AS client,
        i.store_id    AS magasin
FROM    rental    r
JOIN    inventory i ON i.inventory_id = r.inventory_id
JOIN    film      f ON f.film_id      = i.film_id
JOIN    customer  c ON c.customer_id  = r.customer_id;

-- -----

-- 3. PROCEDURE STOCKEE : enregistrer_location (Module 7)
-- -----

-- Recoit des parametres, VERIFIE des conditions (client actif,
-- exemplaire disponible), CALCULE la date de retour prevue,
-- puis ECRIT dans rental. Verrouille l'exemplaire choisi
-- (FOR UPDATE) pour resister a la concurrence (lien Module 8).
DELIMITER $$
CREATE PROCEDURE enregistrer_location(
    IN p_customer_id SMALLINT UNSIGNED,  -- le client
    IN p_film_id     SMALLINT UNSIGNED,  -- le film souhaite
    IN p_store_id    TINYINT  UNSIGNED,  -- le magasin
    IN p_staff_id    TINYINT  UNSIGNED   -- l'employe au guichet
)
BEGIN
    DECLARE v_inventory_id INT DEFAULT NULL;
    DECLARE v_duree        INT DEFAULT NULL;
    DECLARE v_actif        INT DEFAULT 0;

    -- Verification 1 : le client existe et est actif
    SELECT COUNT(*) INTO v_actif
    FROM    customer
```

```

WHERE customer_id = p_customer_id AND active = 1;

IF v_actif = 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Client inconnu ou inactif : location refusee';
END IF;

-- Verification 2 : chercher UN exemplaire libre de ce film
-- dans ce magasin, et le VERROUILLER (FOR UPDATE)
SELECT i.inventory_id, f.rental_duration
INTO v_inventory_id, v_duree
FROM inventory i
JOIN film f ON f.film_id = i.film_id
WHERE i.film_id = p_film_id
    AND i.store_id = p_store_id
    AND NOT EXISTS (SELECT 1 FROM rental r
                     WHERE r.inventory_id = i.inventory_id
                     AND r.return_date IS NULL)

LIMIT 1
FOR UPDATE;

IF v_inventory_id IS NULL THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Aucun exemplaire disponible pour ce film dans ce magasin';
END IF;

-- Ecriture : la location est enregistree
INSERT INTO rental (rental_date, inventory_id, customer_id, staff_id)
VALUES (NOW(), v_inventory_id, p_customer_id, p_staff_id);

-- Calcul restitue a l'appelant : date de retour prevue
SELECT LAST_INSERT_ID() AS rental_id,
       v_inventory_id AS exemplaire,
       DATE_ADD(NOW(), INTERVAL v_duree DAY) AS retour_prevu;
END$$
DELIMITER ;

-----
-- 4. TRIGGERS (Module 7)
-----

-- Trigger 1 : BLOQUE l'incoherence metier – on ne peut pas louer
-- un exemplaire deja sorti (return_date IS NULL). La tentative
-- est journalisee AVANT le refus ; grace a MyISAM, la ligne
-- d'audit survit a l'annulation de l'INSERT fautif.
DELIMITER $$
CREATE TRIGGER trg_bloque_double_location
BEFORE INSERT ON rental
FOR EACH ROW
BEGIN
    DECLARE v_nb INT;

    SELECT COUNT(*) INTO v_nb
    FROM rental
    WHERE inventory_id = NEW.inventory_id
        AND return_date IS NULL;

    IF v_nb > 0 THEN

```

```

        INSERT INTO audit_location (action, inventory_id, customer_id, utilisateur, message)
        VALUES ('LOCATION_REFUSEE', NEW.inventory_id, NEW.customer_id,
                CURRENT_USER(), 'Tentative de louer un exemplaire deja sorti');

        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'REFUSE : cet exemplaire est deja en location';
    END IF;
END$$

-- Trigger 2 : journalise chaque location VALIDEE (piste d'audit).
CREATE TRIGGER trg_audit_location_ok
AFTER INSERT ON rental
FOR EACH ROW
BEGIN
    INSERT INTO audit_location (action, inventory_id, customer_id, utilisateur, message)
    VALUES ('LOCATION_OK', NEW.inventory_id, NEW.customer_id,
            CURRENT_USER(), CONCAT('Location enregistree, rental_id=', NEW.rental_id));
END$$
DELIMITER ;

-----
-- 5. PROCEDURE TRANSACTIONNELLE : louer_et_encaisser (Module 8)
-----

-- Transaction METIER "tout ou rien" : la location ET son paiement
-- sont enregistres ensemble. Si UNE etape echoue, le HANDLER fait
-- ROLLBACK : la base reste coherente (Atomicite du A de ACID).
DELIMITER $$
CREATE PROCEDURE louer_et_encaisser(
    IN p_customer_id SMALLINT UNSIGNED,
    IN p_film_id      SMALLINT UNSIGNED,
    IN p_store_id     TINYINT  UNSIGNED,
    IN p_staff_id     TINYINT  UNSIGNED
)
BEGIN
    DECLARE v_inventory_id INT          DEFAULT NULL;
    DECLARE v_tarif        DECIMAL(4,2) DEFAULT NULL;
    DECLARE v_rental_id    INT          DEFAULT NULL;

    -- En cas d'erreur SQL n'importe ou : tout est annule
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        RESIGNAL; -- on remonte l'erreur d'origine a l'appelant
    END;

    START TRANSACTION;

    -- Choisir et verrouiller un exemplaire libre
    SELECT i.inventory_id, f.rental_rate
    INTO   v_inventory_id, v_tarif
    FROM   inventory i
    JOIN   film f ON f.film_id = i.film_id
    WHERE  i.film_id = p_film_id
        AND i.store_id = p_store_id
        AND NOT EXISTS (SELECT 1 FROM rental r
                        WHERE r.inventory_id = i.inventory_id
                           AND r.return_date IS NULL)

    LIMIT 1

```

```

FOR UPDATE;

IF v_inventory_id IS NULL THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Transaction annulee : aucun exemplaire disponible';
END IF;

-- Ecriture 1 : la location
INSERT INTO rental (rental_date, inventory_id, customer_id, staff_id)
VALUES (NOW(), v_inventory_id, p_customer_id, p_staff_id);
SET v_rental_id = LAST_INSERT_ID();

-- Ecriture 2 : le paiement associe
INSERT INTO payment (customer_id, staff_id, rental_id, amount, payment_date)
VALUES (p_customer_id, p_staff_id, v_rental_id, v_tarif, NOW());

COMMIT;

SELECT v_rental_id AS rental_id, v_tarif AS montant_encaisse,
       'Location + paiement valides ensemble (COMMIT)' AS statut;
END$$
DELIMITER ;

-- -----
-- 6. INDEX (Module 9) – etat FINAL apres optimisation
-- -----
-- NB : Sakila livre deja idx_fk_customer_id sur rental(customer_id).
-- Notre apport est l'index COMPOSITE ci-dessous : il accelere la
-- question centrale du theme "cet exemplaire est-il sorti ?"
-- (WHERE inventory_id = ? AND return_date IS NULL), utilisee par
-- le trigger, la procedure et la vue des locations en cours.
-- Le protocole complet de mesure avant/apres est dans
-- 05_module9_indexation.sql.
CREATE INDEX idx_loc_inventaire_retour ON rental (inventory_id, return_date);

-- -----
-- 7. UTILISATEUR RESTREINT "guichet" (Module 11)
-- -----
-- Principe du MOINDRE PRIVILEGE : le guichetier
-- - LIT les deux vues (jamais les tables de base),
-- - EXECUTE la procedure d'enregistrement,
-- - et RIEN d'autre (aucun DELETE, UPDATE, ni SELECT sur customer).
-- Les vues etant en SQL SECURITY DEFINER (defaut), "guichet"
-- accede aux donnees A TRAVERS elles sans aucun droit direct
-- sur rental/customer/film/inventory.
CREATE USER 'guichet'@'localhost' IDENTIFIED BY 'Guichet#2026!';

GRANT SELECT ON sakila.v_locations_en_cours TO 'guichet'@'localhost';
GRANT SELECT ON sakila.v_historique_locations TO 'guichet'@'localhost';
GRANT EXECUTE ON PROCEDURE sakila.enregistrer_location TO 'guichet'@'localhost';

FLUSH PRIVILEGES;

-- -----
-- FIN – Verification rapide de l'installation
-- -----
SELECT 'Installation terminee' AS statut;
SHOW TRIGGERS LIKE 'rental';

```

```
SELECT COUNT(*) AS locations_en_cours FROM v_locations_en_cours;
```

A.2 — 02_module7_demo.sql

Démonstration du module 7 : vues, procédure enregistrer_location, et trigger bloquant avec remplissage de la table d'audit en direct.

```
-- =====
-- GROUPE 1 – MODULE 7 : DEMONSTRATION EN DIRECT
-- Vues, procedure stockee, trigger bloquant + audit
-- Prerequis : 01_installation.sql execute.
-- Derouler bloc par bloc devant le jury.
-- =====
USE sakila;

-- -----
-- DEMO 1 : LES VUES
-- -----

-- 1a. Locations en cours (avec date de retour prevue calculee)
SELECT * FROM v_locations_en_cours LIMIT 10;

-- 1b. Vue de securite : historique complet SANS email, SANS
--     adresse, SANS montants. Comparer avec la table brute :
SELECT rental_id, date_location, film, client FROM v_historique_locations LIMIT 5;
-- (en tant qu'admin seulement, pour montrer ce que la vue CACHE :)
SELECT customer_id, first_name, last_name, email FROM customer LIMIT 3;

-- -----
-- DEMO 2 : LA PROCEDURE enregistrer_location
-- -----

-- Le client 148 (le plus actif de Sakila) loue le film 1
-- (ACADEMY DINOSAUR) au magasin 1, guichetier n°1.
CALL enregistrer_location(148, 1, 1, 1);
-- => renvoie rental_id, l'exemplaire choisi et le retour prevu.

-- La location apparait immediatement dans la vue :
SELECT * FROM v_locations_en_cours WHERE client = 'ELEANOR HUNT';

-- Cas limite 1 : client inexistant => la procedure REFUSE
CALL enregistrer_location(9999, 1, 1, 1);
-- ERREUR attendue : "Client inconnu ou inactif : location refusee"

-- Cas limite 2 : film sans exemplaire disponible dans ce magasin
-- (le film 802 'SLEEPWALKERS ATTITUDE' n'est en stock nulle part...
-- utiliser la requete ci-dessous pour trouver un cas chez vous)
SELECT f.film_id, f.title
FROM   film f
WHERE  NOT EXISTS (SELECT 1 FROM inventory i WHERE i.film_id = f.film_id)
LIMIT 3;
-- puis :
CALL enregistrer_location(148, /* film_id trouve */ 802, 1, 1);
-- ERREUR attendue : "Aucun exemplaire disponible..."

-- -----
-- DEMO 3 : LE TRIGGER BLOQUANT + AUDIT (le moment cle)
-- -----

-- Etape 1 : trouver un exemplaire ACTUELLEMENT SORTI
SET @inv_sorti := (SELECT inventory_id FROM rental
```

```
WHERE return_date IS NULL LIMIT 1);
SELECT @inv_sorti AS exemplaire_deja_sorti;

-- Etape 2 : TENTER L'ACTION INTERDITE devant le jury –
-- louer cet exemplaire une seconde fois :
INSERT INTO rental (rental_date, inventory_id, customer_id, staff_id)
VALUES (NOW(), @inv_sorti, 148, 1);
-- ERREUR attendue : "REFUSE : cet exemplaire est deja en location"

-- Etape 3 : la table d'audit s'est remplie SOUS LES YEUX du jury
SELECT * FROM audit_location ORDER BY audit_id DESC LIMIT 5;
-- On y voit : la LOCATION_REFUSEE (tentative bloquee) et,
-- plus haut, les LOCATION_OK des demonstrations precedentes.
```

A.3 — 03_module8_transaction.sql

Transaction métier « tout ou rien » : la procédure louer_et_encaisser regroupe location et paiement, avec ROLLBACK automatique en cas d'échec.

```
-- =====
-- GROUPE 1 – MODULE 8 (partie 1) : TRANSACTION METIER
-- "Tout ou rien" : location + paiement dans UNE transaction.
-- Prerequis : 01_installation.sql execute.
-- =====
USE sakila;

-- Etat AVANT : compter les lignes
SELECT (SELECT COUNT(*) FROM rental) AS nb_locations,
       (SELECT COUNT(*) FROM payment) AS nb_paiements;

-- -----
-- CAS 1 : tout se passe bien => COMMIT
-- -----
CALL louer_et_encaisser(148, 2, 1, 1);  -- client 148, film 2 (ACE GOLDFINGER)

-- Les DEUX ecritures ont eu lieu ensemble :
SELECT (SELECT COUNT(*) FROM rental) AS nb_locations,
       (SELECT COUNT(*) FROM payment) AS nb_paiements;

-- -----
-- CAS 2 : une etape echoue => ROLLBACK automatique
-- -----
-- Film sans stock => la transaction est ANNULEE en entier :
CALL louer_et_encaisser(148, 802, 1, 1);
-- ERREUR attendue : "Transaction annulee : aucun exemplaire disponible"

-- Preuve du "tout ou rien" : AUCUNE ligne ajoutée, ni location
-- ni paiement orphelin :
SELECT (SELECT COUNT(*) FROM rental) AS nb_locations,
       (SELECT COUNT(*) FROM payment) AS nb_paiements;

-- -----
-- ACID en une phrase chacune (a dire a l'oral) :
-- A - Atomicite   : la location et le paiement reussissent ou
--                  echouent ENSEMBLE, jamais a moitie.
-- C - Coherence   : la base passe d'un etat valide a un autre
--                  etat valide (pas de paiement sans location).
-- I - Isolation    : deux guichets simultanes ne voient pas les
--                  operations inachevees l'un de l'autre.
-- D - Durabilite   : apres COMMIT, la location survit meme a une
--                  coupure de courant.
-- -----
```

A.4 — 04_module8_concurrence.sql

Mise en scène de la concurrence à deux sessions : le problème (double location sans protection) puis la solution (verrou SELECT ... FOR UPDATE). À dérouler bloc par bloc dans deux terminaux distincts.

```
-- =====
-- GROUPE 1 – MODULE 8 (partie 2) : CONCURRENCE A DEUX SESSIONS
-- Scenario : DEUX GUICHETS louent LE DERNIER EXEMPLAIRE au meme
-- instant. Ouvrir DEUX fenetres (2 onglets phpMyAdmin, ou 2
-- terminaux "mysql -u root -p sakila"). Executer les blocs dans
-- l'ordre indique [A1], [B1], [A2], [B2]...
--
-- IMPORTANT (a expliquer au jury) : le trigger du module 7 ne
-- suffit PAS ici. Sous l'isolation REPEATABLE READ, la session B
-- ne "voit" pas l'INSERT non valide de la session A : les deux
-- verifications passent, et la double location est enregistree.
-- La solution est le VERROU (SELECT ... FOR UPDATE) dans une
-- transaction.
-- =====

-- -----
-- PREPARATION (une seule fois, dans n'importe quelle session)
-- -----

USE sakila;
-- Choisir un exemplaire LIBRE qui servira de "dernier exemplaire" :
SELECT i.inventory_id
FROM   inventory i
WHERE  NOT EXISTS (SELECT 1 FROM rental r
                   WHERE r.inventory_id = i.inventory_id
                        AND r.return_date IS NULL)
LIMIT 1;
-- Noter ce numero et REMPLACER 5 par ce numero dans TOUS les
-- blocs ci-dessous si necessaire.

-- =====
-- PHASE 1 : LE PROBLEME (sans protection => double location)
-- =====

-- [A1] --- SESSION A (guichet 1) -----
USE sakila;
START TRANSACTION;
-- Verification "naive" : l'exemplaire est-il libre ? (lecture simple)
SELECT COUNT(*) AS deja_sorti FROM rental
WHERE  inventory_id = 5 AND return_date IS NULL;      -- => 0 : libre
-- Le guichetier A enregistre la location (client 1) :
INSERT INTO rental (rental_date, inventory_id, customer_id, staff_id)
VALUES (NOW(), 5, 1, 1);
-- NE PAS ENCORE VALIDER : A discute avec son client...

-- [B1] --- SESSION B (guichet 2), PENDANT ce temps -----
USE sakila;
START TRANSACTION;
-- Meme verification naive : B ne voit PAS l'insert non valide de A
SELECT COUNT(*) AS deja_sorti FROM rental
WHERE  inventory_id = 5 AND return_date IS NULL;      -- => 0 aussi !
-- (le trigger fait la meme lecture : il laisse passer, lui aussi)
INSERT INTO rental (rental_date, inventory_id, customer_id, staff_id)
VALUES (NOW(), 5, 2, 2);      -- client 2
```

```

-- [A2] --- SESSION A -----
COMMIT;

-- [B2] --- SESSION B -----
COMMIT;

-- [A3] --- N'importe quelle session : CONSTAT DE L'ANOMALIE -----
SELECT rental_id, customer_id, rental_date
FROM   rental
WHERE  inventory_id = 5 AND return_date IS NULL;
-- => DEUX locations actives pour LE MEME exemplaire physique !
--     La donnee est incoherente : c'est le probleme de concurrence.

-- --- NETTOYAGE avant la phase 2 : annuler la double location -----
-- (desactiver un instant le trigger n'est pas necessaire : on
--  supprime simplement les deux lignes fautives)
DELETE FROM rental WHERE inventory_id = 5 AND return_date IS NULL;

-- =====
-- PHASE 2 : LA SOLUTION (transaction + verrou FOR UPDATE)
-- =====

-- [A1'] --- SESSION A -----
USE sakila;
START TRANSACTION;
-- On VERROUILLE d'abord la ressource disputee (l'exemplaire) :
SELECT inventory_id FROM inventory WHERE inventory_id = 5 FOR UPDATE;
-- Puis verification (lecture verrouillante = elle lit le reel) :
SELECT COUNT(*) AS deja_sorti FROM rental
WHERE  inventory_id = 5 AND return_date IS NULL FOR UPDATE;   -- => 0
-- Libre : A enregistre
INSERT INTO rental (rental_date, inventory_id, customer_id, staff_id)
VALUES (NOW(), 5, 1, 1);
-- (toujours pas de COMMIT : A prend son temps)

-- [B1'] --- SESSION B -----
USE sakila;
START TRANSACTION;
SELECT inventory_id FROM inventory WHERE inventory_id = 5 FOR UPDATE;
-- ==> B RESTE BLOQUEE ICI : elle ATTEND que A libere le verrou.
--     (montrer au jury la fenetre B qui "tourne")

-- [A2'] --- SESSION A -----
COMMIT;   -- A valide et libere le verrou

-- [B2'] --- SESSION B (debloquee automatiquement) -----
-- La verification s'execute MAINTENANT, apres le COMMIT de A :
SELECT COUNT(*) AS deja_sorti FROM rental
WHERE  inventory_id = 5 AND return_date IS NULL FOR UPDATE;   -- => 1 !
-- L'exemplaire n'est plus libre : le guichet B RENONCE proprement :
ROLLBACK;
-- (et si B s'obstinait a faire l'INSERT, le trigger du module 7,
--  qui lit désormais la location validee de A, le REFUSERAIT et
--  journaliserait la tentative dans audit_location.)

-- [B3'] --- CONSTAT FINAL -----
SELECT rental_id, customer_id FROM rental

```

```
WHERE inventory_id = 5 AND return_date IS NULL;
-- => UNE SEULE location active : l'anomalie a disparu.

-- Nettoyage de fin de demo :
DELETE FROM rental WHERE inventory_id = 5 AND return_date IS NULL;
```

A.5 — 05_module9_indexation.sql

Protocole complet de mesure avant / après indexation, avec EXPLAIN et chronométrage (SET profiling).

```
-- =====
-- GROUPE 1 – MODULE 9 : INDEXATION ET OPTIMISATION
-- Protocole complet AVANT / APRES, reproductible devant le jury.
--
-- POINT CLE A EXPLIQUER : Sakila livre DEJA des index sur rental
-- (idx_fk_customer_id, idx_fk_inventory_id) car MySQL exige un
-- index pour chaque cle etrangere. Pour mesurer honnetement
-- l'AVANT, on retire d'abord ces index (donc les FK qui s'y
-- appuient), on mesure, on cree NOS index, on remet les FK.
-- =====
USE sakila;

-- -----
-- ETAPE 0 : preparation de l'etat "SANS INDEX"
-- -----
-- (si 01_installation.sql a deja cree l'index composite, on le
-- retire aussi ; ignorer l'erreur s'il n'existe pas)
ALTER TABLE rental DROP INDEX idx_loc_inventaire_retour;

ALTER TABLE rental DROP FOREIGN KEY fk_rental_customer;
ALTER TABLE rental DROP INDEX idx_fk_customer_id;
ALTER TABLE rental DROP FOREIGN KEY fk_rental_inventory;
ALTER TABLE rental DROP INDEX idx_fk_inventory_id;

-- Activer le chronometrage (MariaDB/MySQL) :
SET profiling = 1;

-- =====
-- MESURES "AVANT" – executer chaque requete 3 fois et retenir
-- la moyenne (SHOW PROFILES en fin de bloc)
-- =====

-- ----- Requete R1 : toutes les locations d'un client ----
EXPLAIN SELECT * FROM rental WHERE customer_id = 148;
-- Lire dans le plan : type = ALL, rows ~ 16044
-- => la base PARCOURT TOUTE LA TABLE (full scan) car aucun index
-- ne permet d'accéder directement aux lignes du client.
SELECT SQL_NO_CACHE * FROM rental WHERE customer_id = 148;
SELECT SQL_NO_CACHE * FROM rental WHERE customer_id = 148;
SELECT SQL_NO_CACHE * FROM rental WHERE customer_id = 148;

-- ----- Requete R2 : cet exemplaire est-il sorti ? -----
-- (la question centrale du theme : trigger, procedure, vue)
EXPLAIN SELECT * FROM rental
WHERE inventory_id = 367 AND return_date IS NULL;
-- type = ALL egalement : full scan.
SELECT SQL_NO_CACHE * FROM rental WHERE inventory_id = 367 AND return_date IS NULL;
SELECT SQL_NO_CACHE * FROM rental WHERE inventory_id = 367 AND return_date IS NULL;
SELECT SQL_NO_CACHE * FROM rental WHERE inventory_id = 367 AND return_date IS NULL;

-- ----- Requete R3 : locations non rendues (jointure) -----
EXPLAIN SELECT c.last_name, f.title, r.rental_date
FROM rental r
```

```

JOIN customer c ON c.customer_id = r.customer_id
JOIN inventory i ON i.inventory_id = r.inventory_id
JOIN film f ON f.film_id = i.film_id
WHERE r.return_date IS NULL AND r.customer_id = 148;
SELECT SQL_NO_CACHE c.last_name, f.title, r.rental_date
FROM rental r
JOIN customer c ON c.customer_id = r.customer_id
JOIN inventory i ON i.inventory_id = r.inventory_id
JOIN film f ON f.film_id = i.film_id
WHERE r.return_date IS NULL AND r.customer_id = 148;

-- ----- Requete R4 : historique d'un exemplaire -----
EXPLAIN SELECT * FROM rental WHERE inventory_id = 100 ORDER BY rental_date;
SELECT SQL_NO_CACHE * FROM rental WHERE inventory_id = 100 ORDER BY rental_date;

-- >>> RELEVER LES TEMPS "AVANT" :
SHOW PROFILES;

-- =====
-- ETAPE 2 : CREATION DES INDEX ADAPTES
-- =====
-- Index simple : colonne du WHERE de R1
CREATE INDEX idx_rental_customer ON rental (customer_id);

-- Index COMPOSITE : couvre R2 et R4 (prefixe inventory_id) et
-- accelere le filtre return_date IS NULL. L'ordre des colonnes
-- compte : la colonne d'egalite d'abord, le filtre ensuite.
CREATE INDEX idx_loc_inventaire_retour ON rental (inventory_id, return_date);

-- Remettre les clees etrangeres d'origine (elles reutilisent nos
-- nouveaux index : pas d'index en double) :
ALTER TABLE rental ADD CONSTRAINT fk_rental_customer
    FOREIGN KEY (customer_id) REFERENCES customer (customer_id)
    ON DELETE RESTRICT ON UPDATE CASCADE;
ALTER TABLE rental ADD CONSTRAINT fk_rental_inventory
    FOREIGN KEY (inventory_id) REFERENCES inventory (inventory_id)
    ON DELETE RESTRICT ON UPDATE CASCADE;

-- =====
-- MESURES "APRES" – memes requetes, memes conditions
-- =====
EXPLAIN SELECT * FROM rental WHERE customer_id = 148;
-- Lire : type = ref, key = idx_rental_customer, rows ~ 46
-- => ACCES DIRECT par l'index au lieu du parcours complet.
SELECT SQL_NO_CACHE * FROM rental WHERE customer_id = 148;
SELECT SQL_NO_CACHE * FROM rental WHERE customer_id = 148;
SELECT SQL_NO_CACHE * FROM rental WHERE customer_id = 148;

EXPLAIN SELECT * FROM rental
WHERE inventory_id = 367 AND return_date IS NULL;
-- type = ref, key = idx_loc_inventaire_retour, rows ~ 1
SELECT SQL_NO_CACHE * FROM rental WHERE inventory_id = 367 AND return_date IS NULL;
SELECT SQL_NO_CACHE * FROM rental WHERE inventory_id = 367 AND return_date IS NULL;
SELECT SQL_NO_CACHE * FROM rental WHERE inventory_id = 367 AND return_date IS NULL;

EXPLAIN SELECT c.last_name, f.title, r.rental_date
FROM rental r
JOIN customer c ON c.customer_id = r.customer_id

```

```

JOIN inventory i ON i.inventory_id = r.inventory_id
JOIN film f      ON f.film_id      = i.film_id
WHERE r.return_date IS NULL AND r.customer_id = 148;
SELECT SQL_NO_CACHE c.last_name, f.title, r.rental_date
FROM rental r
JOIN customer c  ON c.customer_id = r.customer_id
JOIN inventory i ON i.inventory_id = r.inventory_id
JOIN film f      ON f.film_id      = i.film_id
WHERE r.return_date IS NULL AND r.customer_id = 148;

EXPLAIN SELECT * FROM rental WHERE inventory_id = 100 ORDER BY rental_date;
SELECT SQL_NO_CACHE * FROM rental WHERE inventory_id = 100 ORDER BY rental_date;

-- >>> RELEVER LES TEMPS "APRES" :
SHOW PROFILES;

-- =====
-- TABLEAU COMPARATIF A REMPLIR (a recopier dans le dossier)
-- =====
-- | Requete | Temps avant (ms) | Plan avant | Temps apres (ms) | Plan apres |
Gain |
--
|-----|-----|-----|-----|-----|
----|
-- | R1      | ...          | ALL (scan) | ...          | ref idx_rental_customer |
x ... |
-- | R2      | ...          | ALL (scan) | ...          | ref idx_loc_inv_retour   |
x ... |
-- | R3      | ...          | ALL sur r   | ...          | ref (jointures indexees) |
x ... |
-- | R4      | ...          | ALL (scan) | ...          | ref idx_loc_inv_retour   |
x ... |
--
-- NB : sur 16 044 lignes les temps sont en millisecondes ; c'est
-- le PLAN (ALL -> ref) et le nombre de lignes examinees
-- (16044 -> ~46 ou ~1) qui prouvent le gain, et qui montrent que
-- sur des millions de lignes l'ecart deviendrait enorme.

```

A.6 — 06_module11_securite.sql

Démonstration de la sécurité : droits de l'utilisateur « guichet », puis sauvegarde, simulation de sinistre et restauration.

```
-- =====
-- GROUPE 1 – MODULE 11 : SECURITE ET ADMINISTRATION
-- Utilisateur restreint "guichet" + sauvegarde / restauration.
-- Prerequis : 01_installation.sql (l'utilisateur y est cree).
-- =====

-- -----
-- PARTIE A – VERIFIER LES DROITS (en tant qu'ADMIN)
-- -----

SHOW GRANTS FOR 'guichet'@'localhost';
-- Attendu : SELECT sur les 2 vues, EXECUTE sur la procedure,
-- et RIEN d'autre. C'est le principe du MOINDRE PRIVILEGE.

-- -----
-- PARTIE B – DEMONSTRATION EN TANT QUE "guichet"
-- -----

-- Se connecter dans une AUTRE session :
-- Terminal      : mysql -u guichet -p          (mdp : Guichet#2026!)
-- phpMyAdmin    : se deconnecter puis se connecter en "guichet"
-- Puis executer :

-- B1. La vue FONCTIONNE (mais sans email, adresse, montants) :
SELECT * FROM sakila.v_locations_en_cours LIMIT 5;
SELECT * FROM sakila.v_historique_locations LIMIT 5;

-- B2. L'accès aux TABLES DE BASE est REFUSE :
SELECT * FROM sakila.customer;
-- ERREUR attendue : SELECT command denied to user 'guichet'@...

SELECT * FROM sakila.rental;
-- ERREUR attendue : SELECT command denied

-- B3. Toute ECRITURE directe est REFUSEE :
DELETE FROM sakila.rental WHERE rental_id = 1;
-- ERREUR attendue : DELETE command denied

UPDATE sakila.customer SET first_name = 'X' WHERE customer_id = 1;
-- ERREUR attendue : UPDATE command denied

-- B4. Mais le guichetier PEUT enregistrer une location, car il
-- passe par la PROCEDURE (execution avec les droits du createur) :
CALL sakila.enregistrer_location(148, 3, 1, 1);
-- => la seule voie d'écriture autorisée, contrôlée et audité.

-- -----
-- PARTIE C – SAUVEGARDE / PERTE / RESTAURATION (en ADMIN)
-- -----

-- C1. SAUVEGARDE (à lancer dans un terminal Windows/Linux,
-- PAS dans phpMyAdmin) :
--
-- mysqldump -u root -p --routines --triggers sakila > sakila_sauvegarde.sql
--
-- (--routines : inclut les procedures ; --triggers : les triggers.
```

```
-- Alternative phpMyAdmin : onglet Exporter > methode
-- personnalisee > cocher "Procedures et fonctions" et "Triggers".)

-- C2. RELEVER L'ETAT DE REFERENCE avant le sinistre :
SELECT COUNT(*) AS nb_paiements_avant FROM sakila.payment; -- ex. 16049

-- C3. SIMULER LA PERTE DE DONNEES (le "sinistre") :
DELETE FROM sakila.payment;
SELECT COUNT(*) AS nb_paiements_apres_sinistre FROM sakila.payment; -- 0 !

-- C4. RESTAURER (terminal) :
--
-- mysql -u root -p sakila < sakila_sauvegarde.sql
--
-- (Alternative phpMyAdmin : onglet Importer > choisir le fichier.)

-- C5. VERIFIER que l'etat initial est retabli :
SELECT COUNT(*) AS nb_paiements_restares FROM sakila.payment; -- retour a la valeur de C2
SELECT * FROM sakila.v_locations_en_cours LIMIT 3; -- les vues fonctionnent

-- -----
-- MATRICE DES DROITS (a mettre dans le dossier)
-- -----
```

Objet	root (admin)	guichet
Tables rental/customer..	Tous droits	AUCUN acces
v_locations_en_cours	Tous droits	SELECT seulement
v_historique_locations	Tous droits	SELECT seulement
enregistrer_location	Tous droits	EXECUTE seulement
louer_et_encaisser	Tous droits	AUCUN acces
DELETE / UPDATE / DROP	Oui	NON (refuse)
Donnees sensibles (email, adresse, montants)	Visibles	JAMAIS visibles